FIT2099 A2 Design Documentation

[Contribution Log]

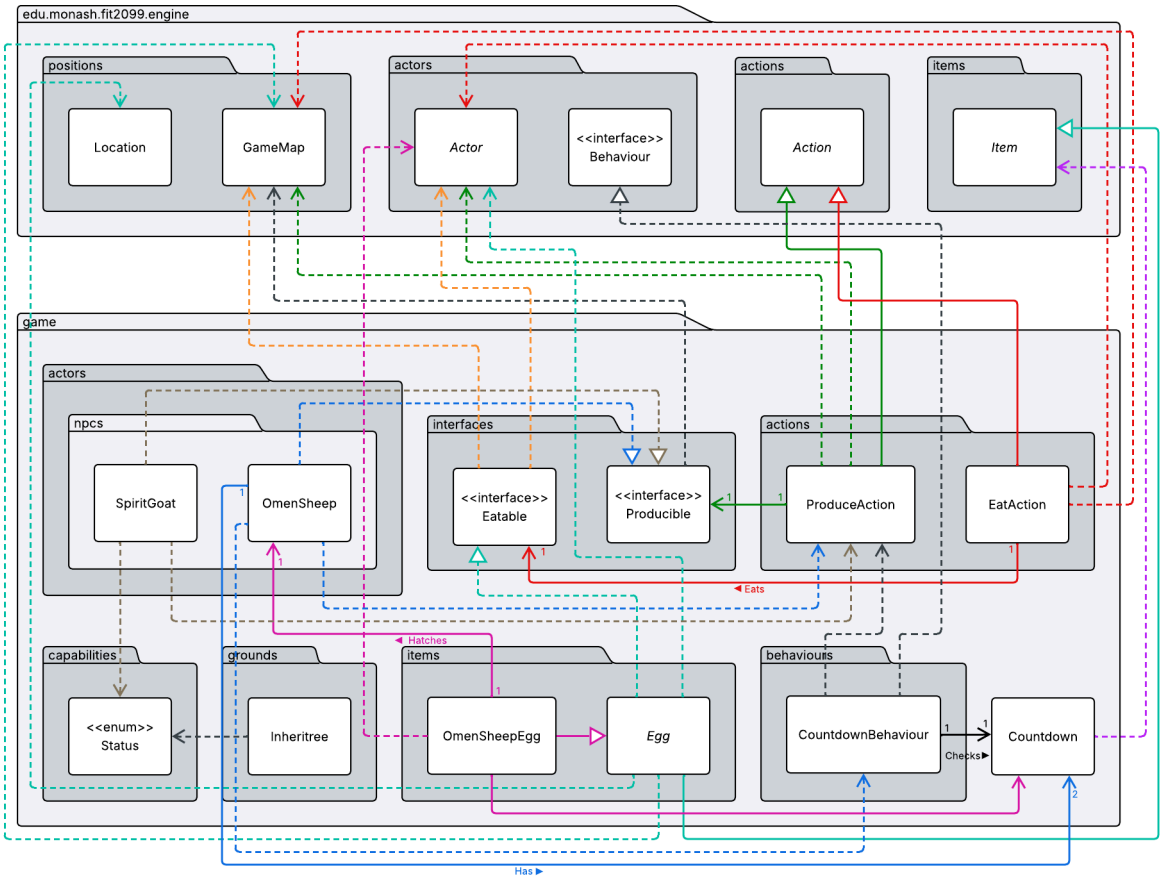
Changes from A1

As per feedback from A1, we made a few improvements to the base code structure:

- **Introduction of abstract NPC class** containing protected behaviours, including WanderBehaviour, eliminating the need to repeat/duplicate behaviour implementation for subsequent NPCs
- **Extended NPC class with an AttackableNPC abstract class** which defines NPCs that can be attacked by the "Farmer"
- These architectural changes provide better adherence to SOLID principles:
    - **OCP**: The hierarchy is open for extension but closed for modification
    - **DIP**: Code depends on abstractions (NPC, AttackableNPC) rather than concrete implementations
    - **DRY**: Common behavior is defined once in the parent classes

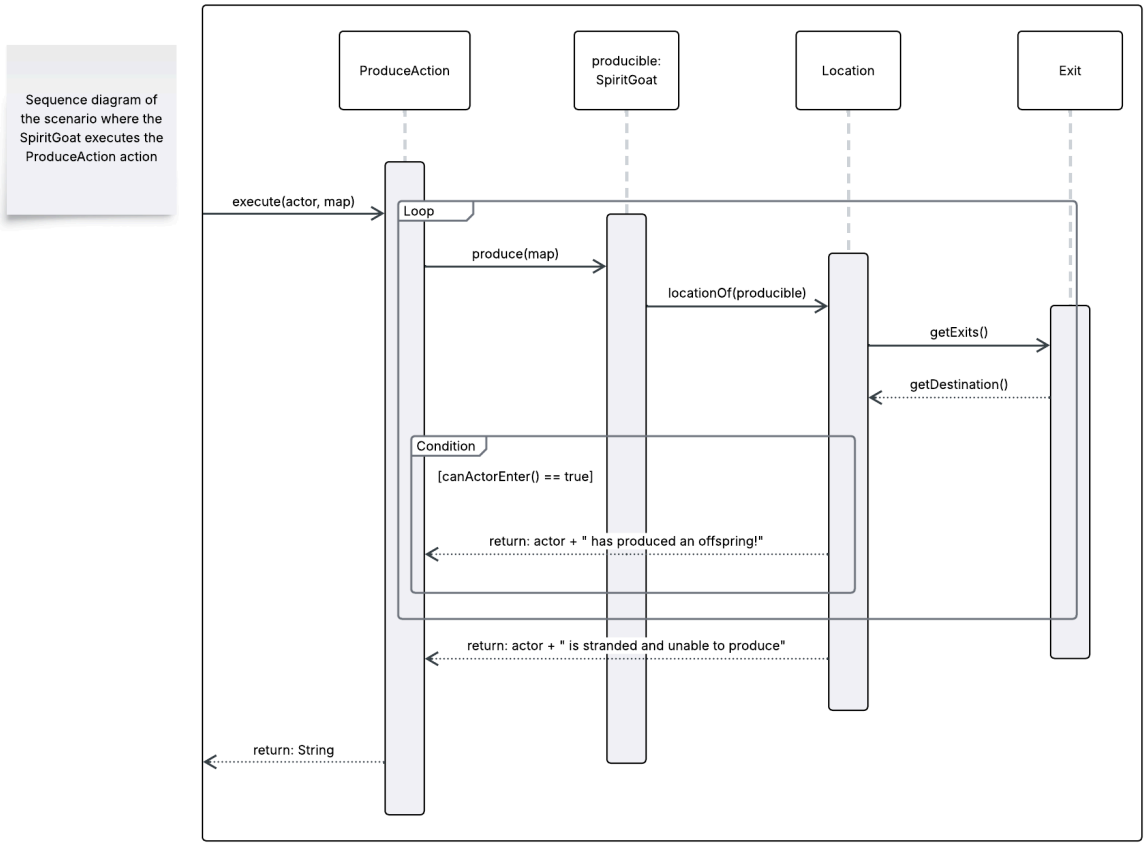
REQ 1

UML Class Diagram:



This UML diagram represents an extension to Elden Thing where the SpiritGoat and OmenSheep are able to produce offspring of a SpiritGoat and an Egg respectively. The Egg is also able to hatch, as well as be carried and eaten by the Player.

ProduceAction Sequence Diagram:



ProduceAction Class:

The `Producible` interface is an attribute of the `ProduceAction` class, and is what the methods and behaviours of the `ProduceAction` class are carried out on.

- By using a `Producible` interface, `ProduceAction` is able to work without implementing individual classes, which adheres to the **Dependency Inversion Principle (DIP)**
- `ProduceAction` has a `Producible` attribute which allows all producible to have `ProduceAction` as part of their action list, following the **Open/Closed Principle (OCP)**
- The `ProduceAction` class is solely responsible for the action of an `Actor` producing, which follows the **Single Responsibility Principle (SRP)**

EatAction Class:

The `EatAction` class contains an `Eatable` interface as an attribute, which the methods and behaviours are acted on.

- As the `EatAction` class is responsible for the action of an `Eatable` being eaten by an `Actor`, it follows the **Single Responsibility Principle (SRP)**
- `EatAction` utilises an `Eatable` interface instead of using individual classes, which follows the **Dependency Inversion Principle (DIP)**
- `EatAction` can handle and build upon any `Eatable` without extra modification, which adheres to the **Open/Closed Principle (OCP)**

Egg Abstract Class:

The `Egg` class is an abstract class extending the `Item` class, and implements the `Eatable` interface which allows it to be eaten and have `EatAction` as part of its action list. In addition, the `Egg` is also able to hatch through a `hatch()` method.

`Egg` was originally implemented as a regular class for the `OmenSheep` rather than an abstract class. However, it was later changed to an abstract class as it was determined to follow SOLID principles better, and an additional `OmenSheepEgg` class was added.

Pros of Changing Egg to an Abstract Class:

- Avoids repeating code in future implementations of other `Egg` classes, such as the `hatch()` method or adding `EatAction` to the action list - **Don't Repeat Yourself (DRY)**
- By making `Egg` an abstract class, it is easier to extend with subclasses, which adheres to the **Open/Closed Principle (OCP)**
- By depending on an abstract `Egg` class instead of a concrete one, coupling is reduced which creates more flexible code, following the **Dependency Inversion Principle (DIP)**

Cons of Changing Egg to an Abstract Class:

- As it is uncertain whether what requirements future subclasses will have, they may inherit some unnecessary functionality, which risks the **Interface Segregation Principle (ISP)**

Hatch Method:

The hatch() method in `Egg` was originally implemented as a separate `HatchAction` class, as it was originally presumed to work similarly to other actions. However, it was later changed to be a method completed inside the `Egg` class itself.

Benefits to Removing the HatchAction Class:

- The `Egg` is an Item, which means it should not be able to perform an `Action` like an `Actor` is able to. By keeping its responsibilities within the responsibilities of an Item, it follows the **Single Responsibility Principle (SRP)**
- As `Egg` is a subclass of `Item` and items aren't able to perform actions, the `Egg` shouldn't either. Doing so would violate the **Liskov Substitution Principle (LSP)** by making the `Egg` class non-substitutable

OmenSheepEgg Class:

The `OmenSheepEgg` is an extension of `Egg`, which means it doesn't need to implement the `Eatable` interface as it is inherited. It hatches into an `OmenSheep` after it is on the ground for 3 turns. It also restores health if consumed by the player.

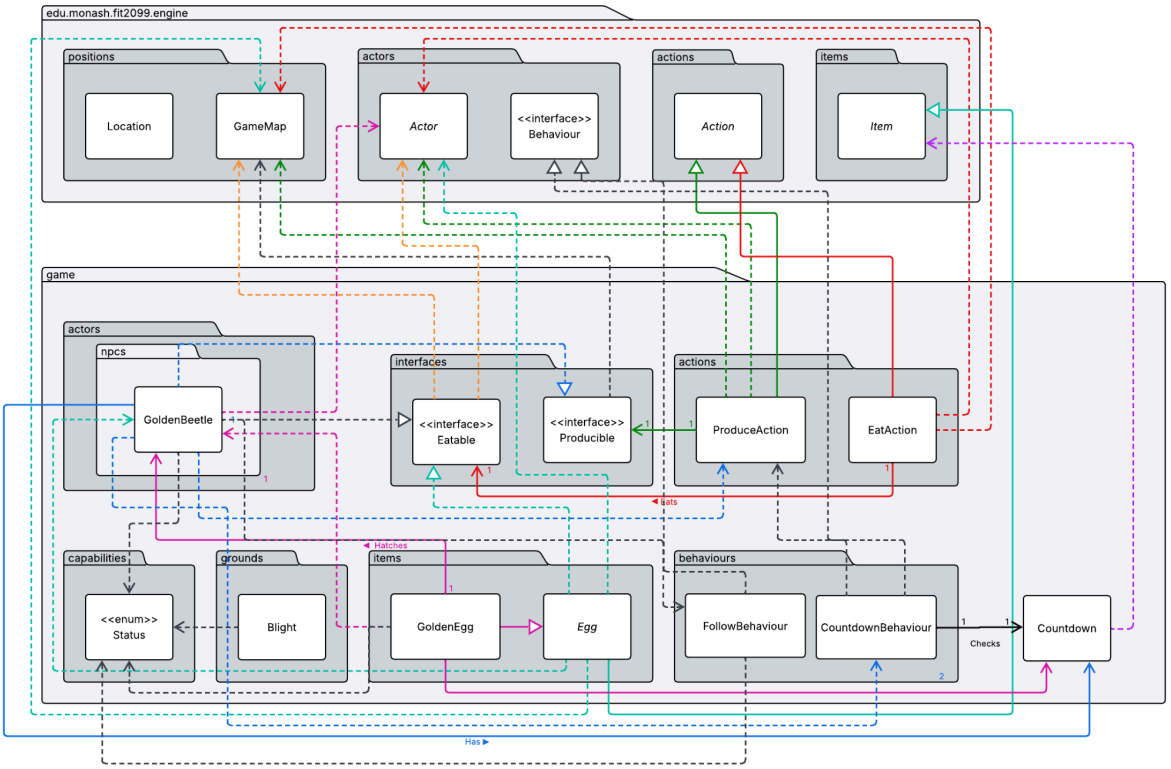
- It utilises a `Countdown` attribute to keep track of the turns until hatching, which reuses methods rather than repeating code - **Don't Repeat Yourself (DRY)**
- It has an `OmenSheep` attribute that represents the hatched form of the `OmenSheepEgg` which allows the hatched form to be easily accessed without creating new classes for it - **Don't Repeat Yourself (DRY)**

Status.BLESSED_BY_GRACE:

The `SpiritGoat` produces offspring if they have something in their surroundings that is blessed by grace such as the `Inheritree`. Rather than having the `SpiritGoat` check each blessed object individually, it instead checks objects with the `BLESSED_BY_GRACE` status, such as the `Inheritree`.

- This allows new blessed objects to be added in the future without modification, following the **Open/Closed Principle (OCP)**

REQ 2



GoldenBeetle Class

The `GoldenBeetle` class encapsulates the life cycle and behavior of a non-hostile NPC that interacts dynamically with the world by producing `GoldenEggs`, following nearby actors, and being consumable by the player. The design follows SOLID principles and aims to promote modularity, reusability, and future extensibility.

SRP (Single Responsibility Principle):
The class focuses solely on encapsulating the Golden Beetle's behavior (egg production, following, being eaten). Supporting features such as countdowns and item consumption are abstracted into interfaces and utility classes.

OCP (Open/Closed Principle):
New behaviors (e.g., new actions, interaction rules) can be added without modifying existing code. For example, CountdownBehaviour delegates expiry handling to an injected `Action`, allowing future countdown-based events (not just egg-laying) to be reused without altering the GoldenBeetle.

DRY (Don't Repeat Yourself):
Countdown logic for turn-based events is centralised in the `Countdown` class and

`CountdownBehaviour`, avoiding duplication across entities like the OmenSheep and GoldenBeetle. The Eatable and Producible interfaces provide shared contracts for consumption and spawning behavior, respectively.

LSP (Liskov Substitution Principle):
All beetles and eggs adhere to interfaces (`Eatable`, `Producible`), allowing them to be used interchangeably wherever those interfaces are expected.

ISP (Interface Segregation Principle):
By breaking behavior into granular interfaces (e.g., `Eatable`, `Producible`), each class implements only what it needs. This promotes clarity and keeps responsibilities separate.

GoldenEgg Class

The `GoldenEgg` inherits from `Egg` and implements the `Eatable` interface. It hatches into a `GoldenBeetle` only when placed near cursed terrain. It also restores stamina if consumed by the player.

Design Highlights:

- Uses `SurroundingCapabilityCondition` utility to check for cursed tiles.
- Ensures mutual exclusivity between hatching and consumption (via `canHatch` flag).
- The egg lifecycle is consistent with game mechanics and extensible for future variations (e.g., different hatching conditions).

Countdown + CountdownBehaviour

The `Countdown` class encapsulates time-based expiry logic and is decoupled from entity implementations. `CountdownBehaviour` allows NPCs to trigger specific `Actions` (e.g., `ProduceAction`) when the countdown expires.

Benefits:

- Promotes reuse of time-based behaviors (e.g., hatching, self-destruction, special attacks).
- Encapsulates state transitions (e.g., reset and expire) in a maintainable way.

EatAction & ProduceAction

These generic action classes are injected into behavior or interface-driven triggers and carry out their logic on `Eatable` or `Producible` entities.

Adheres to:

- SRP: Each class does one thing—consume or produce.
- OCP: Additional effects or logging can be added without altering the actor/item logic.

- DIP: Actions depend on abstractions (`Eatable`, `Producible`), not concrete classes.

Design Alternatives Considered

In an earlier design, egg production logic was handled directly in `playTurn()`, leading to tight coupling and duplication of countdown logic between entities. This was refactored into `CountdownBehaviour` to:

- Improve code reuse
  Make behavior declarative rather than imperative
- Enable other NPCs to use similar countdown-based behaviors without rewriting logic

Advantages

- High modularity and separation of concerns
- Easily extensible to new egg types, hatch conditions, or countdown-based behaviors
- Interfaces make contracts explicit and promote code reuse

Disadvantages

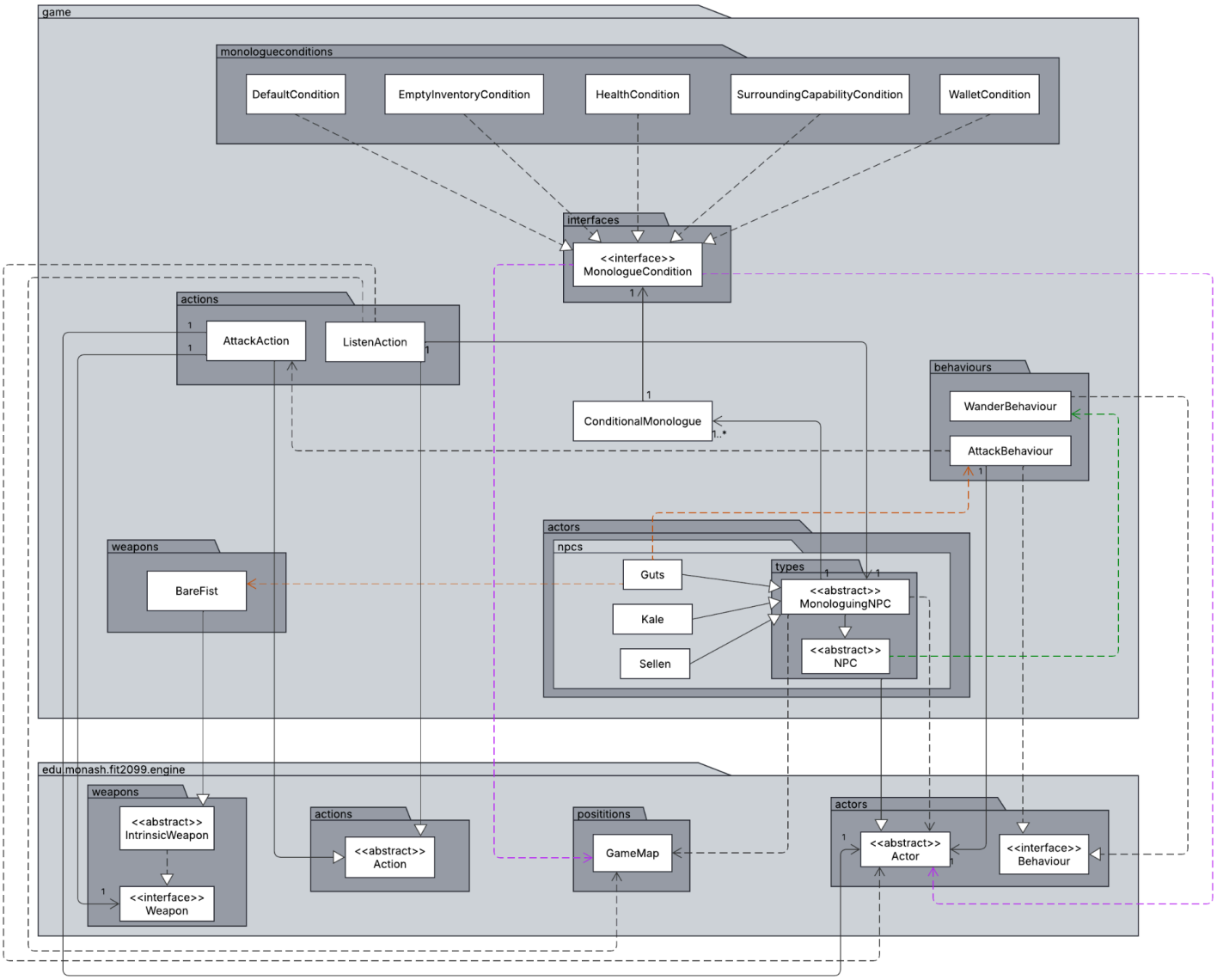
- Slight increase in complexity due to abstracted behaviors and state management
- Requires careful initialization of injected dependencies (e.g., correct action passed to `Countdown`)

Maintainability & Extensibility

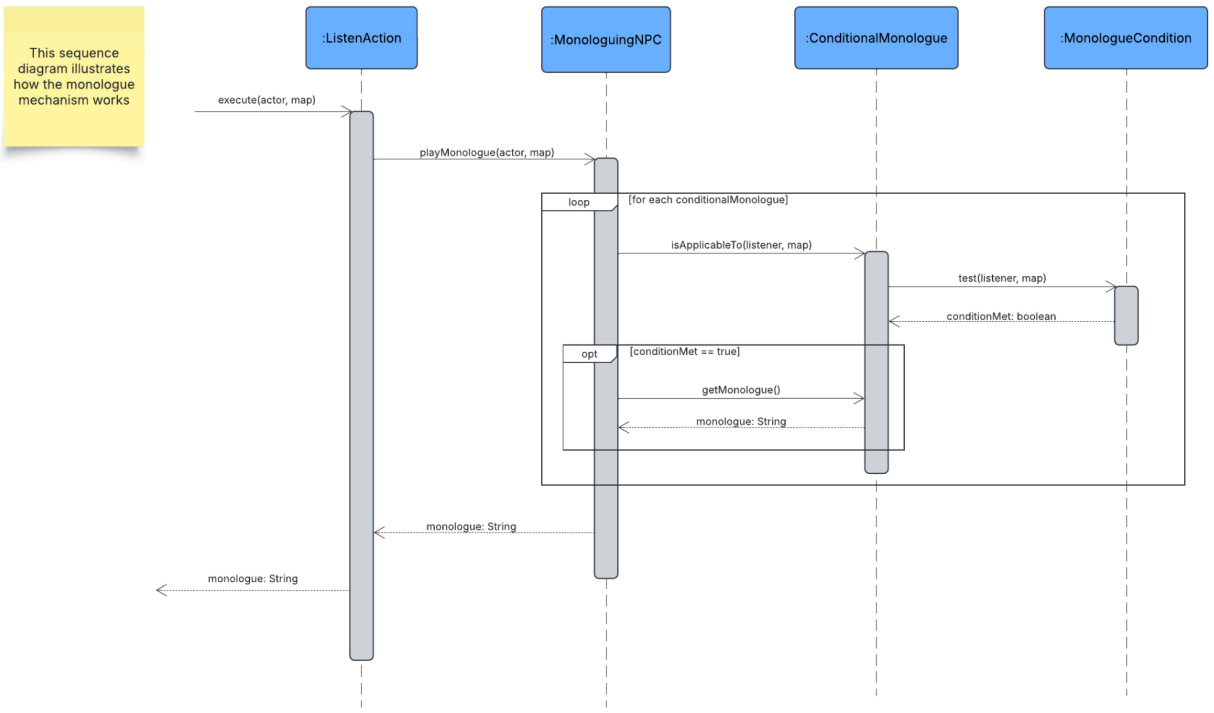
This design allows new entities (e.g., cursed-hatching lizards, time-triggered healers) to reuse countdown and production logic with minimal duplication. If a new requirement introduced multi-phase hatching or conditional follow behavior, the current setup supports that via behavioural composition.

Future additions like alternative hatching triggers (e.g., turns, blessings) or more egg types can be introduced by implementing `Producible`, extending `Egg`, or composing new `Countdown` actions—without modifying existing code.

REQ 3



The UML diagram represents an extension to the Elden Thing game architecture that allows the “Farmer” to listen to NPCs Sellen, Kale and Guts. This system introduces a flexible framework for NPCs to deliver contextual monologues based on player attributes and game state.



The sequence diagram illustrates how the monologue mechanism works, detailing the interactions between ListenAction, Monologuer, ConditionalMonologue and MonologueCondition.

MonologuingNPC Abstract Class

The design centres around the MonologingNPC abstract class implemented by NPCs that can provide monologues. This approach adheres to several SOLID principles:

- **SRP:** The class defines a focused contract specifically for monologue-related behaviour without forcing unrelated methods on implementing classes
- **LSP**: The class is an extension of NPC class which also extends Actor and can be used wherever Actor is used, ensuring type substitutability throughout the inheritance hierarchy
- **OCP**: New Monologuer types can be added without modifying existing code
- **DRY**: Parent implementation of `playMonologue()` and `selectRandomMonologue()` methods prevents code duplication across implementing classes

The default implementations handle the logic for filtering applicable monologues based on conditions and selecting one randomly, allowing implementing classes to focus solely on defining their monologue pools.

In an earlier design iteration, monologues were dynamically added within each NPC's implementation of the `playMonologue()` method. This approach was rejected for several reasons:

- **Violation of OCP**: Each time a new condition for monologues needed to be added, the NPC class implementation would require modification, making the system difficult to extend
- **Code Duplication**: Similar conditional logic for selecting monologues was repeated across different NPC implementations, violating DRY principles
- **Tight Coupling**: Direct embedding of monologue selection logic within NPC classes created tight coupling between character behaviour and dialogue systems

The revised design addresses these issues by extracting the monologue functionality into a dedicated class with implementations for common behaviour, allowing NPC classes to focus on defining their monologue pools rather than implementing selection logic.

ConditionalMonologue Class

ConditionalMonologue serves as the composition element that pairs a specific condition with its corresponding dialogue text. This design choice:

- Adheres to **SRP** by focusing solely on pairing conditions with monologues
- Creates a clear separation between monologue content and the conditions that trigger them
- Enables NPCs to have multiple monologues with different triggering conditions

The 1:1 association between Monologuer and a list of ConditionalMonologues establishes a flexible structure where NPCs can have varied responses based on different player states.

MonologueCondition Interface

The MonologueCondition interface establishes a contract for different types of conditions that can trigger monologues. This design:

- Follows **DIP** by depending on abstractions rather than concrete implementations
- Adheres to **OCP** as new condition types can be added without modifying existing code
- Enables **LSP** as any MonologueCondition can be used where a condition is expected

The `test()` method provides a standardised way to evaluate if a condition applies to a specific listener in the current game context.

Concrete MonologueCondition Implementations

The DefaultCondition, EmptyInventoryCondition, HealthCondition, SurroundingCapabilityCondition and WalletCondition classes represent concrete implementations of MonologueCondition that trigger monologues based on the player's state.

Advantages:

- Follows **SRP** as each class focuses on its specific monologue condition
- Clear separation of concerns between monologue content and triggering conditions
- Highly extensible system where new NPCs and condition types can be added without modifying existing code
- Reusable condition types that can be shared across different NPCs
- Default implementations in the interface reduce code duplication
- Composition-based approach avoids multilevel inheritance issues

Disadvantages:

- Increased complexity compared to direct implementation in NPC classes
- Potential overhead from creating multiple objects for the monologue system
- Each NPC must manage its own monologue pool, which could lead to maintenance challenges if many NPCs implement the Monologuer interface

ListenAction Class

The ListenAction class serves as the bridge between the player and Monologuer interface, enabling dialogue to be initiated with nearby NPCs.

- Association with Monologuer interface rather than specific NPC classes allows for interaction with any character implementing the Monologuer interface, adhering to **DIP** and **LSP**
- **SRP** is maintained as ListenAction is singularly responsible for enabling speaking/listening interactions between the player and NPCs

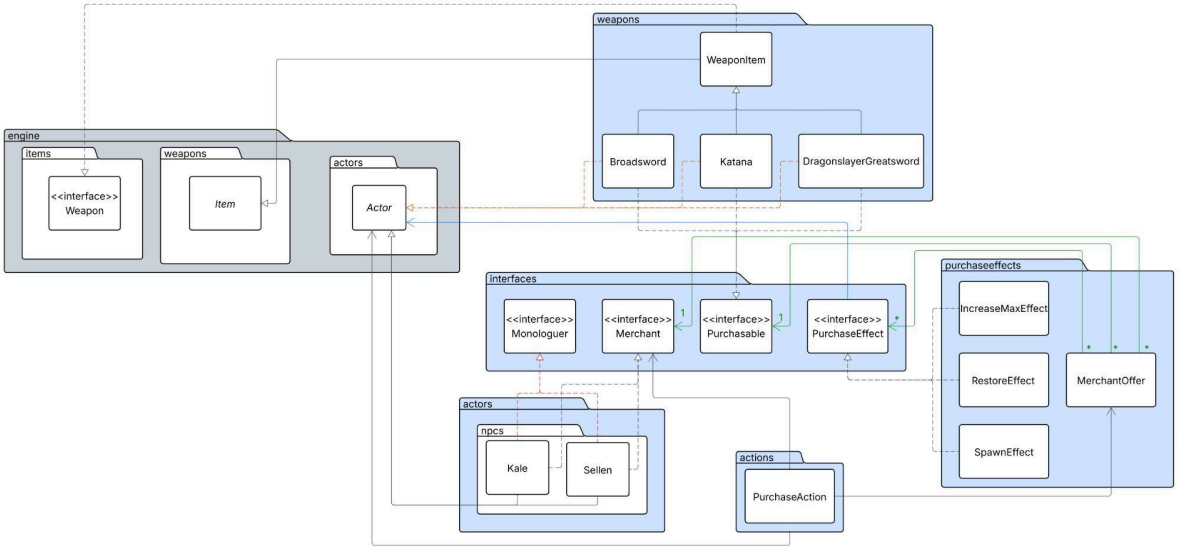
AttackBehaviour Implementation

The introduction of AttackBehaviour specifically for the Guts NPC represents an important extension to the NPC behaviour model:

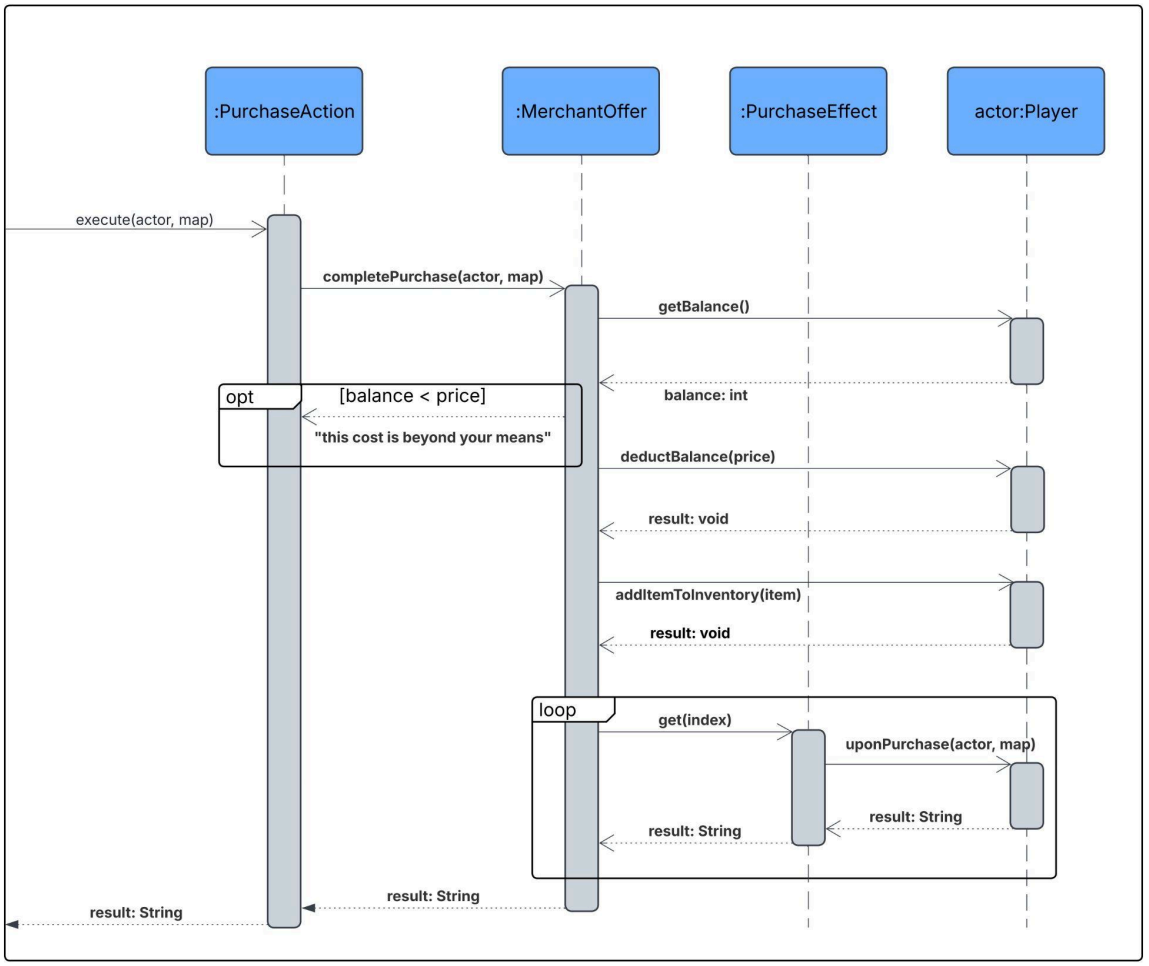
- Positioned with highest priority in Guts' behaviors map, ensuring it takes precedence over other behaviours
- Adheres to **SRP** by isolating attack-related behaviour in a dedicated class rather than embedding it within the NPC class

- Follows **ISP** by implementing only the necessary methods for calling AttackAction without forcing unrelated functionality

REQ 4



Sequence Diagram



PurchaseAction

The PurchaseAction class to encapsulate the logic of purchasing an item, ensuring that the action is clearly defined and easily extendable.
Pros:

- The class clearly defines the purchase action, making it easy to understand and maintain.
- New purchase-related features can be added without modifying existing code.
- The class can be reused across different parts of the application where purchase actions are needed.

Cons:

- Depends on MerchantOffer, which might limit flexibility if the offer structure changes significantly.

Single Responsibility Principle (SRP): Has a single responsibility to handle the action of purchasing an item. It encapsulates the logic required to execute a purchase and describe it in a menu, adhering to SRP.

Open/Closed Principle (OCP): Open for extension but closed for modification. If new types of purchases or additional logic are needed, they can be added by extending the class or modifying the MerchantOffer without altering the existing PurchaseAction class.

DRY: Avoids repetition by using the MerchantOffer to handle the specifics of the purchase, ensuring that the purchase logic is centralized and not duplicated.

Purchasable Interface

The PurchaseEffect interface allows for different effects to be applied upon purchase, providing flexibility and extensibility.
Pros:

- Different effects can be implemented and applied without modifying existing code.
- New effects can be added easily by implementing the interface.
- The interface promotes reuse of effect logic across different purchase scenarios.

Cons:

- Managing multiple effects might increase complexity, especially if effects interact with each other.
- Each effect requires a separate implementation, which might be excessive for simple effects.

Interface Segregation Principle (ISP): Ensures that any item that can be purchased implements the uponPurchase method. This keeps the interface focused and specific, adhering to ISP by not forcing classes to implement methods they don't use.

DRY Principle: By defining a common interface for purchasable items, the code avoids duplication of purchase-related methods across different item classes.

PurchaseEffect Interface and Different Purchase Effects

The interface allows for different effects to be applied upon purchase, providing flexibility and extensibility.
Pros:

- Flexibility: Different effects can be implemented and applied without modifying existing code.
- Extensibility: New effects can be added easily by implementing the interface.
- Reusability: The interface promotes reuse of effect logic across different purchase scenarios.

Cons:

- Managing multiple effects might increase complexity, especially if effects interact with each other.
- Each effect requires a separate implementation, which might be excessive for simple effects.

Open/Closed Principle (OCP): The PurchaseEffect interface allows for different purchase effects to be implemented without modifying existing code. New effects can be added by implementing this interface.
Liskov Substitution Principle (LSP): Any class implementing PurchaseEffect can be used interchangeably, ensuring that the system behaves correctly regardless of the specific effect applied.
DRY Principle: Centralizing the effect logic in a single interface prevents code duplication and promotes reuse across different purchase scenarios.

Merchant Interface

The Merchant interface ensures that all merchants provide a common method for retrieving offerings, promoting consistency and flexibility.
Pros:

- Consistency: All merchants adhere to a common interface, ensuring consistent behavior.
- Flexibility: New merchant types can be added easily by implementing the interface.
- Reusability: The interface promotes reuse of merchant logic across different merchant classes.

Cons:

- The interface is limited to offering-related methods, which might require additional interfaces for other merchant behaviors.
- Each new merchant type must implement the interface, which might be redundant for simple merchants.

Interface Segregation Principle (ISP): The interface is focused on providing offerings, ensuring that implementing classes only need to provide this specific functionality.
DRY Principle: By defining a common interface for merchants, the code avoids duplication of merchant-related methods across different merchant classes.

MerchantOffer Class

The MerchantOffer class to encapsulate the details of a purchase offer, ensuring that the offer logic is centralised and easily extendable.
Pros:

- Clarity: The class clearly defines the offer logic, making it easy to understand and maintain.
- Extensibility: New offer-related features can be added without modifying existing code.
- Reusability: The class can be reused across different parts of the application where offers are needed.

Cons:

- The class depends on Purchasable and PurchaseEffect, which might limit flexibility if these structures change significantly.

Single Responsibility Principle (SRP): Responsible for managing the details of a purchase offer, including the item, price, and effects. It encapsulates all related logic, adhering to SRP.
Open/Closed Principle (OCP): The class is designed to be extended with new types of offers or effects without modifying existing code.
Liskov Substitution Principle (LSP): The class can be used in place of any other offer-related class, provided it adheres to the expected interface.
DRY Principle: The class centralizes the logic for completing a purchase, ensuring that this logic is not duplicated elsewhere in the codebase.

Weapons (Katana, Broadsword & DragonslayerGreatsword)

Weapon classes encapsulate the attributes and behaviors of different weapons, ensuring that each weapon is clearly defined and easily extendable.
Pros:

- Clarity: Each weapon class clearly defines the weapon's attributes and behaviors, making it easy to understand and maintain.
- Extensibility: New weapons can be added easily by creating new classes or extending existing ones.
- Reusability: Common weapon logic can be abstracted into a base class or interface, promoting reuse across different weapon classes.

Cons:

- Managing multiple weapon classes might increase complexity, especially if weapons have many unique attributes or behaviors.

Single Responsibility Principle (SRP): Each weapon class should have a single responsibility, such as defining the weapon's attributes and behavior.
Open/Closed Principle (OCP): New weapons can be added by creating new classes that extend a common weapon class or implement a weapon interface, without modifying existing weapon classes.
Liskov Substitution Principle (LSP): Each weapon class should be usable in place of any other weapon class, provided they adhere to a common interface or superclass.

DRY Principle: Common weapon attributes and behaviors should be abstracted into a base class or interface to avoid duplication across different weapon classes.